Advanced database management systems
Topic 2: Replication and log shipping

In the loose coupling case study of Topic 1 we were in fact implementing a special form of *replication*.

Replication concepts and architecture

Replica means a copy of the original. In database technology, replication is used to automate the copying and merging of data from or to multiple sources. The components of the replication metaphor are as follows.

- The **publisher** is the entity (a database server) that has data to be shared. Such data is organized into **publications**. Each publication contains one or more **articles**. The articles can be tables or parts of tables, stored procedures or other database objects.
- The **subscriber** is the entity that subscribes to publications. It can be the same database server as the publisher or another server. Several subscribers, possibly on different servers, may subscribe to the same publication.
- The **subscription** may have various modalities with respect to the way and scheduling of copying. It can also include data filters or on-the-fly data transform steps. It can be a **push** or a **pull** subscription. The pull subscriptions are created at, and scheduled by, the subscriber.

The main types and application scenarios of replication are as follows.

- **Snapshot** replication. After an initial snapshot of the articles, the copied objects will be dropped and re-created on the subscriber each time the data is refreshed, regardless of whether there was any change in the publication. Applicable for copying parts of an OLTP database off to a data warehouse or a reporting server, scheduled out of office hours. An example is sending data generated during the day overnight ('point in time reports'). Since several subscriptions may point to the same destination database, replication can be used as an ETL (extract-transform-load) mechanism if SQL Server technology is used by all publishers. WARNING: due to the drop/re-create mechanism, objects at the subscriber may be temporarily inaccessible. The latency of the data must also be tolerated. *All other replication types below are initialized with a snapshot*.
- **Transactional** replication. It copies only the data that has been changed, and it can be configured for near real time data synchronization. A primary key on the replicated tables is needed. Applicable when considerable latency is a problem and when we do not want to move unchanged data. An example is the off-site branches of a company that have their own local servers holding only parts of the central database relevant for their operation. Such an architecture improves site autonomy and robustness of the information system.
- **Merge** replication. In this scheme the subscribers may themselves generate changes to the data and there is a mechanism in place that distributes these changes to all parties and merges them into a consistent database. The merging process may also involve conflict resolution. In order to identify records across multiple servers, the tables must have a field of UNIQUEIDENTIFIER data type with ROWGUIDCOL[1] property. A typical scenario is the case of traveling businesspeople who are not always connected the central database. The changes they make on their local database is merged with others' changes automatically.

The type of the replication is always determined by the publication.

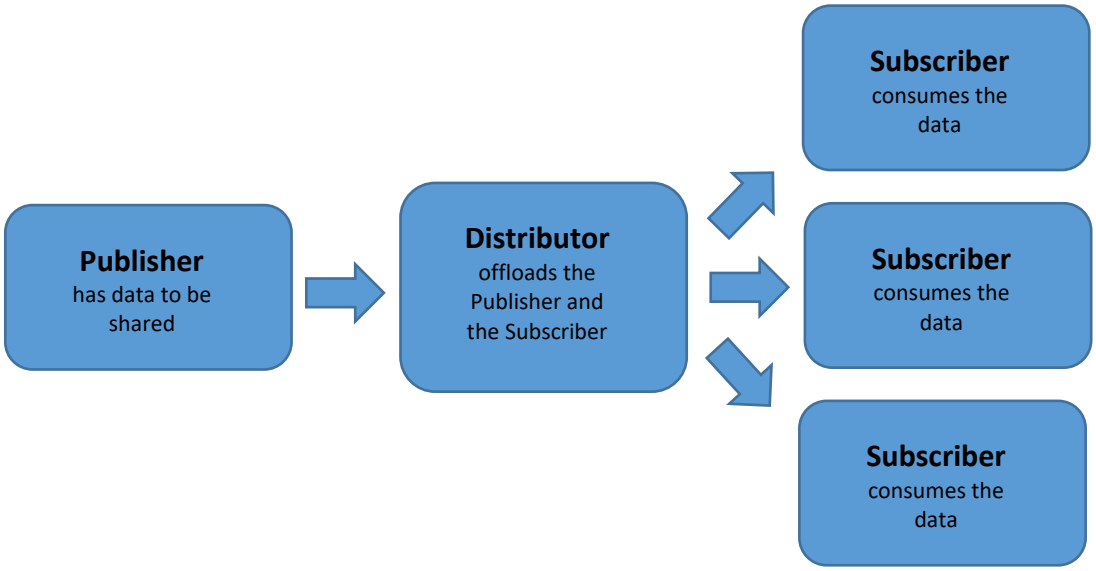
[1] Works like an identity column without a seed and with globally unique values.

Replication technology is based on scheduled jobs and, in the case of merge replication, triggers on published articles.

Replication is **not recommended** when an exact copy of a whole database is to be maintained on a remote server and the goal is to improve availability and reliability, because log shipping and the Always-On technology of SQL server 2012 and later offer a simpler and more robust solution.

WARNING: though replication prepares multiple copies of the data, it is not a replacement for backups and disaster recovery planning.

There are three **server roles** in replication, the *publisher*, the *distributor* and the *subscriber*. All three roles may be taken by the same server instance when a local database is replicated into another local database, or by different instances. In more realistic setups, the distributor role is taken by another server to offload the publisher. The **distributor** is responsible for storing the changed data in a shared folder or a distribution database and for forwarding the data to the subscribers. A publisher can have only one distributor, but a distributor may serve several subscribers.



In **bi-directional** or **updatable** replication the subscriber may be allowed to make changes on the publisher as well.

SQL server implements replication functionality with various **agents**. These agents are jobs running under the supervision of SQL Server Agent.

- **The Snapshot agent** generates the snapshot and stores it in the **snapshot folder** at the distributor. The agent uses the bcp (bulk copy) utility to copy the articles of the publication.
- **The Distribution agent**. In snapshot replication this agent applies the snapshot to the subscriber and in transactional replication it runs the transactions held in the **distribution database** on the subscriber. The distribution database is a system database on the distributor, therefore you can find it in the System Databases group. This agent runs at the subscriber for pull subscriptions and it runs at the publisher for push subscriptions.
- **The Log reader agent** reads the transaction log at the publisher and copies the relevant transactions from the log to the distribution database. It is used only in transactional replication. There is a separate agent for each database published.

- **The Queue reader agent** copies changes made by the subscribers to the publisher in an *updatable* or *bi-directional* transactional replication.
- **The Merge agent** merges incremental changes that occur at both the subscriber and the publisher in merge replication. Detecting changes is based on triggers. The merge agent is not used in transactional replication.

Except for a pull subscription, all agents run at the distributor.

Snapshot replication

First of all, configure the test environment. For the replication examples to work as expected, you need three 'named' MS SQL Server instances installed on the same server machine. They should be named PRIM, SECOND and THIRD.

Scenario: we want to replicate the orders of the American customers to another database on the same server (Principal) to refresh the reporting data warehouse overnight. We choose snapshot replication.

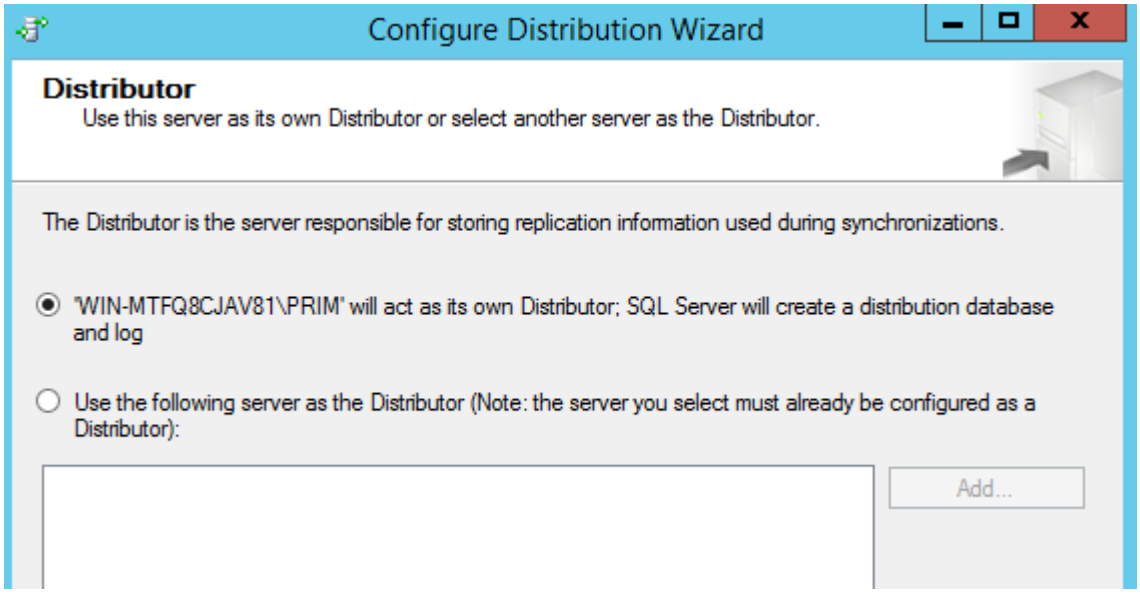
Creating the publication

1. Connect to the Principal server and create a new database called Northwind if it does not exist. Run the create dump of the Northwind database.
2. In order to avoid the error "*Cannot execute as the database principal because the principal "dbo" does not exist, …*" etc. that is due to an incomplete restore of the Northwind database, execute the following script on the Northwind database:

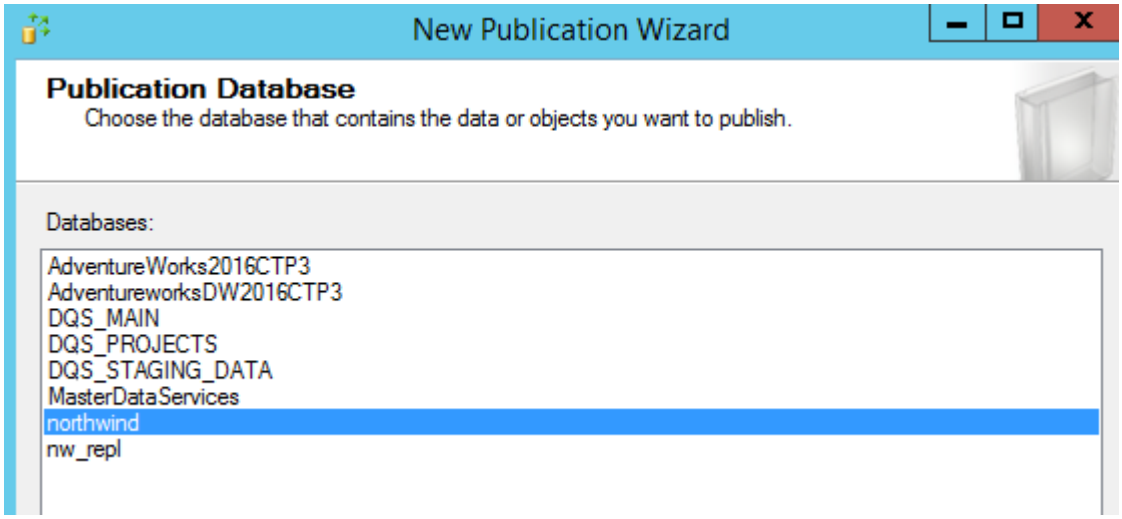
```
EXEC sp_changedbowner 'sa';

ALTER AUTHORIZATION ON DATABASE::northwind TO sa;
```

3. Create another empty database called nw_repl, also on the PRIM server.
4. Start SQL Server Agent if it is not running
5. Selecting Replication -> Configure Distribution, set the PRIM to be its own distributor. Note that the snapshot folder will be this:

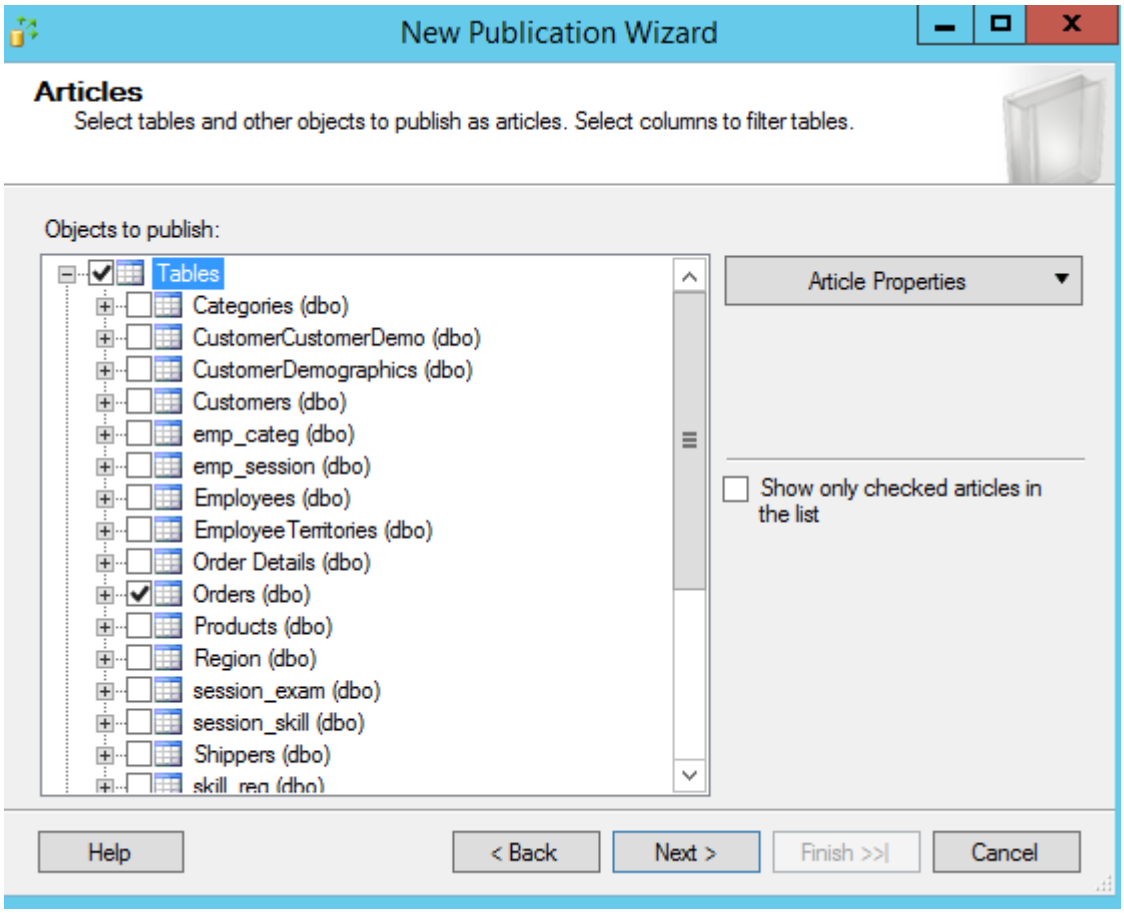
   C:\Program Files\Microsoft SQL Server\MSSQL14.PRIM\MSSQL\ReplData



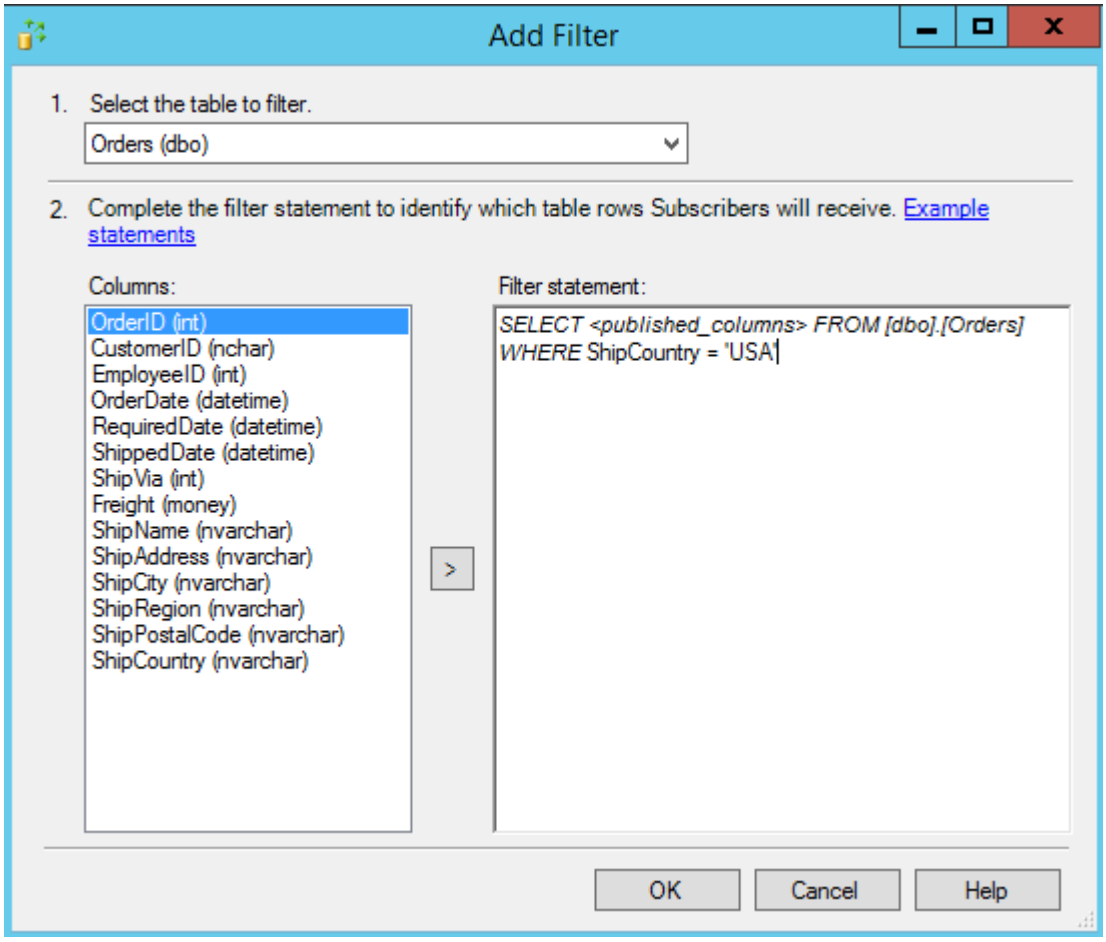
6. Start the New publication wizard and select northwind as the publication database:


New Publication Wizard
Publication Database
Choose the database that contains the data or objects you want to publish.
Databases:
AdventureWorks2016CTP3
AdventureworksDW2016CTP3
DQS_MAIN
DQS_PROJECTS
DQS_STAGING_DATA
MasterDataServices
northwind
nw_repl

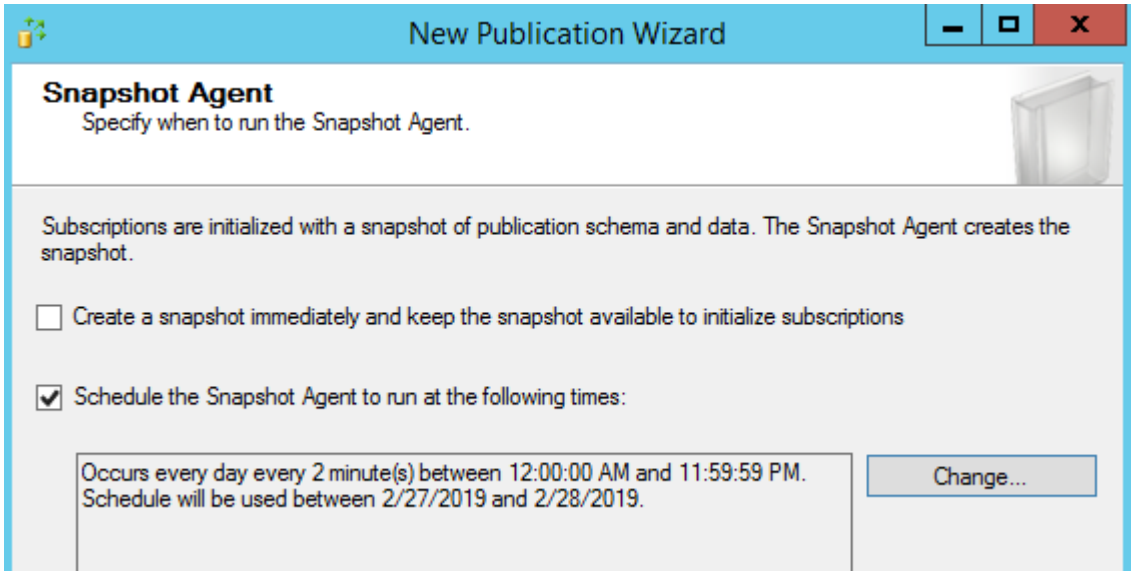
7. On the next panel, select Snapshot publication, then select the Orders table as the single article of the publication:



8. On the Filter table dialog, choose Add
9. Complete the filter statement to filter out American customers

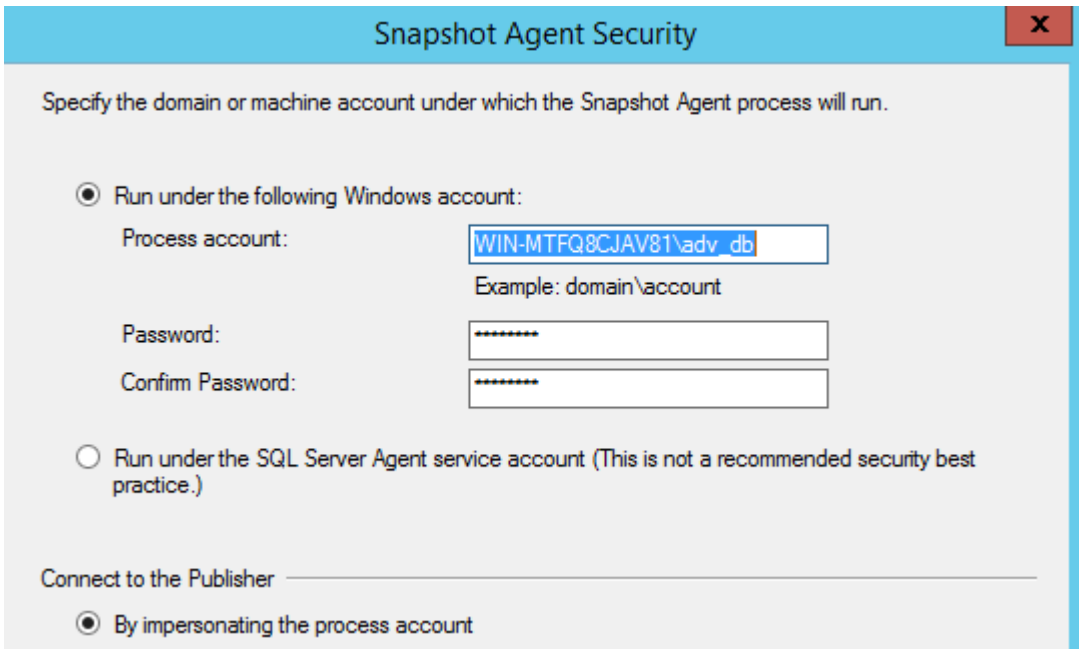


10. Specify that the snapshot agent should run every two minutes by selecting Change on the next panel. Note: *We use this short time interval* ***only for demo*** *purposes. The agent runs the* bcp *utility which places a lock on the whole table until it finishes the copying in order to guarantee data consistency. This means the blocking of all other transactions that may wish to modify the table. Snapshot generation in production systems should be scheduled considering performance implications.*



11. We now have to specify agent security. On the Security settings tab, set your own user credentials and impersonate process account. This is the simplest way of ensuring that the

snapshot agent will have write permission to the snapshot folder. **Note**: you might wonder why the SQL Server Agent service or the SQLSERVER service uses a low privilege non-administrator Windows account. The reason for this is that in this way an attacker who has successfully cracked the DBMS has less chance to corrupt the whole server.



12. On the next panel, select create the publication and name it 'orders'.

Checking the publication

The new publication appears under Local publications. The snapshot folders are created at C:\Program Files\Microsoft SQL Server\MSSQL14.PRIM\MSSQL\repldata\unc\WIN-MTFQ8CJAV81$PRIM_NORTHWIND_TEST, but no actual snapshot was not generated because no subscriptions needed initialization yet.

Check the new job that appears under SQL Server Agent jobs. The job history shows that the agent is run periodically as configured.

Creating a push subscription

1. Start the new subscription wizard from the pop-up menu on the orders publication. Select the orders publication as the source
2. On the next panel, choose Run all agents at the distributor for push subscription
3. Specify the same server as the subscriber and the nw_repl as the subscription database
4. Set the security of the distribution agent the same way as the snapshot agent
5. For the schedule, select Run continuously to provide minimal latency
6. On the next panel, select initialization. This will generate the first snapshot in the distribution folder.
7. Finish creating the new subscription

Note: you can change the properties of subscriptions and publications later if you select Properties from their pop-up menu.

Checking the subscription

1. Locate the new Orders table in the nw_repl database and check that it contains the USA orders
2. Check the contents of the snapshot folder. **Bulk copy** is a fast method SQL server uses to insert data directly into database files.

Name	Date modified	Type
Orders_2.bcp	4/4/2018 11:16 AM	SQL Server Replication Snapshot Bulk-copy Data File
Orders_2.idx	4/4/2018 11:16 AM	SQL Server Replication Snapshot Index Script
Orders_2.pre	4/4/2018 11:16 AM	PRE File
Orders_2.sch	4/4/2018 11:16 AM	SQL Server Replication Snapshot Schema Script

3. Start the Replication monitor from the pop-up menu of the new subscription and check the publication and the subscription status. You can also review the active replication agents here:

Publications | Subscription Watch List | Agents

Agent types: Maintenance jobs

Status	Job	Last Start Time	Duration	Last Action
Never started	Reinitialize subscr...			
Not running	Agent history clea...	4/4/2018 11:20:...	00:00:00	The job succeed...
Not running	Replication agent...	4/4/2018 11:20:...	00:00:00	The job succeed...
Never started	Expired subscripti...			
Not running	Distribution clean ...	4/4/2018 11:25:...	00:00:00	The job succeed...
Never started	Replication monit...			

8. Open the Job activity monitor. The distribution agent appears as a new job in the list, with a status of Executing all the time
9. Change the first USA record of the orders table at the publisher with an UPDATE statement. The change appears in the replicated table shortly (ca. 30 seconds) after the snapshot agent is run the next time.
10. Finally, delete the subscription and the publication. This can be accomplished by selecting Generate scripts form the popup menu of the Replication group, specifying 'To drop…' and running the script in an editor. Alternatively, you can delete the objects one-by-one manually.
11. The replicated tables at the subscriber will not be deleted by deleting the subscription, so delete the Orders table manually from the nw_repl database.

Transactional replication

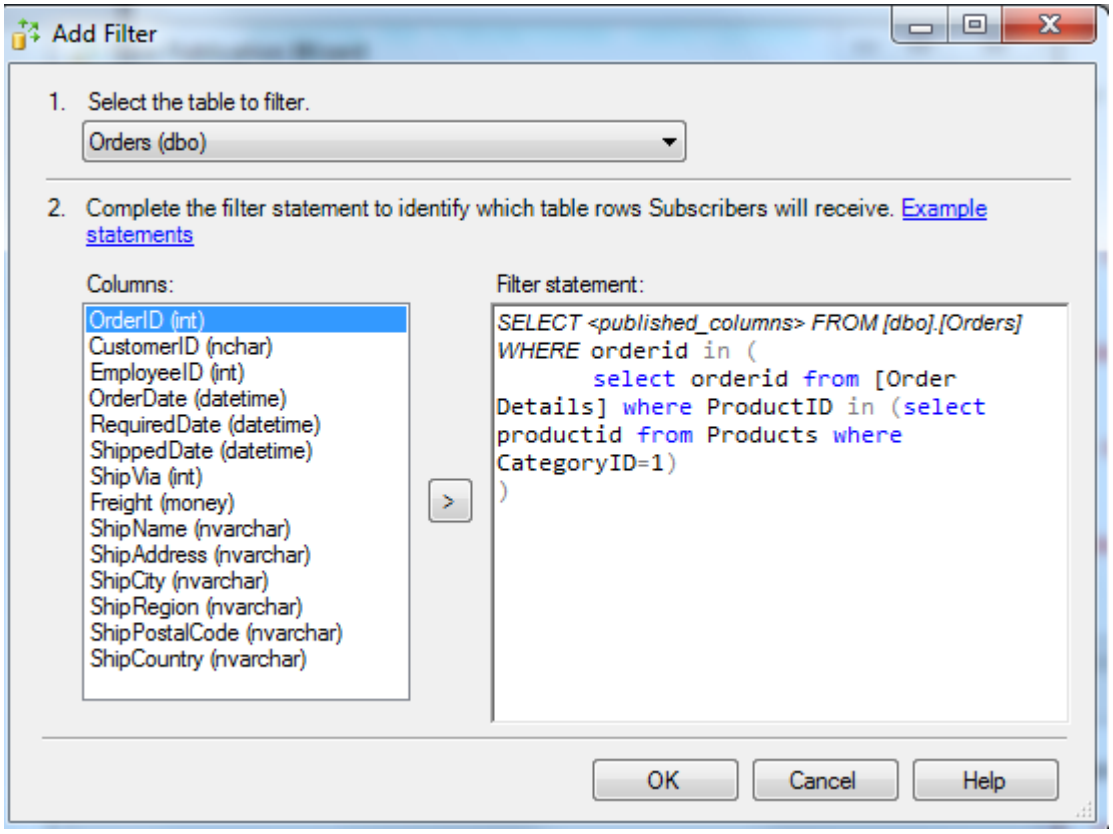
Scenario: we wish to create a near-real time (scheduled) loose coupling between the central Northwind database and an off-site division that deals only with the products of the category 'Beverages' (CategoryID=1). We replicate only orders and order items that are beverages via transactional replication.

First we implement this demo on a single server (Principal). The filter condition above can be defined as follows:

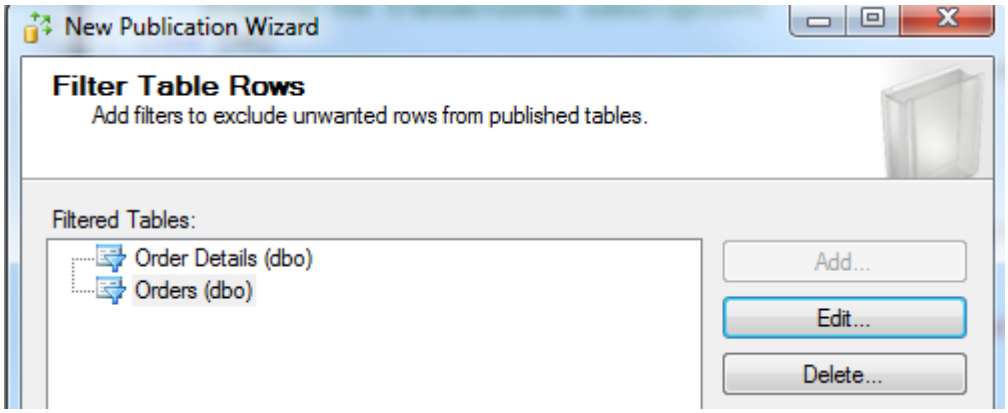
```
select * from [Order Details] where ProductID in (select productid from Products where CategoryID=1)
```

```
select * from orders where orderid in (
        select orderid from [Order Details] where ProductID in (select productid from Products where CategoryID=1)
)
```

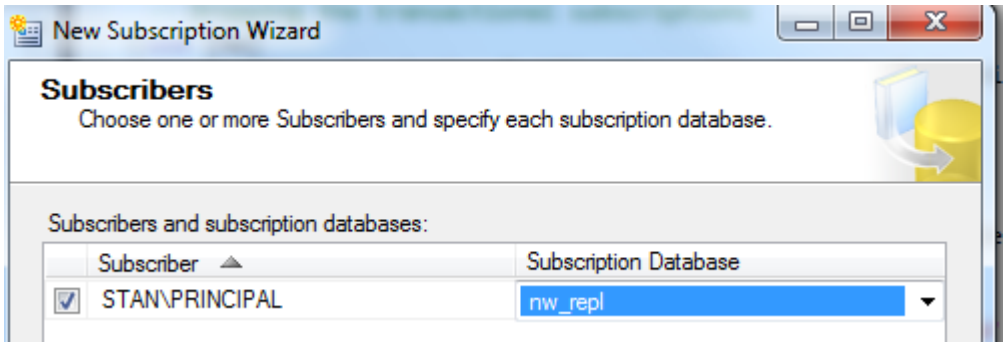
1. You can reset the replication configuration by selecting Disable publishing and distribution from the Replication menu
2. Define the type of the publication as transactional on the Publication type dialog panel
3. Select the Orders and Order Details tables on the Articles panel
4. On the Filter table panel, add the filter to the two tables one by one by copying the WHERE part from the above queries. For the Orders table:



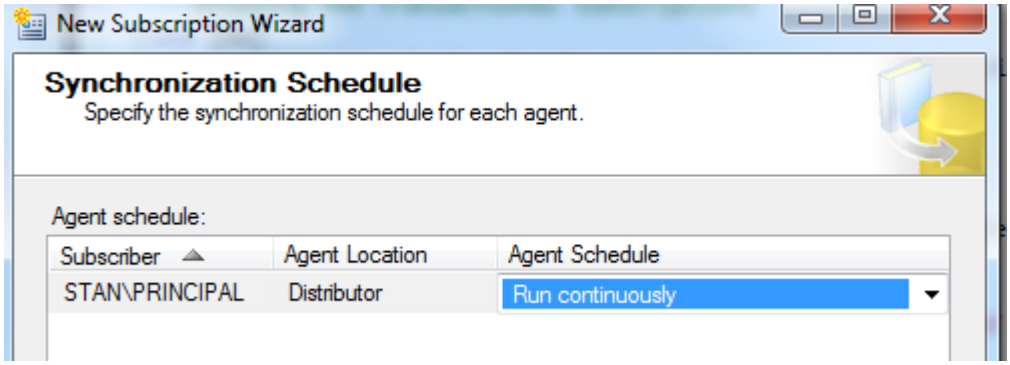
5. You should have the filters defined for both tables:



6. On the next panel, select Create a snapshot immediately
7. On the panels that follow set the security of the agents the same way as in the previous demo
8. Name the publication nw_trans and finish creating the publication
9. Select New subscription in the pop-up menu of the publication
10. Select the Run all agents at the Distributor on the next panel (push subscription)
11. Select the nw_repl database as the subscription database:



12. Set the security of the distribution agent the same way as in the previous demo
13. Specify Run continuously for the schedule of the Log reader and Distribution agents:



14. On the next panel titled Initialize subscriptions, choose Immediately
15. Test the correct operation of the transactional replication. Update the employeeID in the first record of the Orders table in the northwind database and then select the same record in the nw_repl database. You should see the changed value within 10 seconds.
16. Check the operation of the replication agents

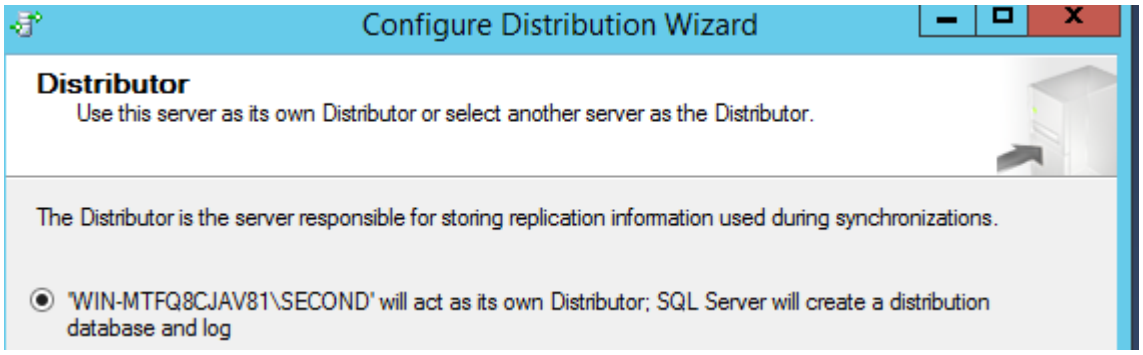
17. Clean up the replication by deleting all replication objects

Replication between separate servers[2]

In the next demo we implement the same transactional replication in a more realistic scenario using the PRIM as the publisher, the SECOND server as the distributor and the THIRD as the subscriber, respectively. In an even more realistic scenario, they would be not only separate server instances, but they would also reside on separate server machines. We cannot, however, implement such a scenario in the lab.

Configuring the distributor

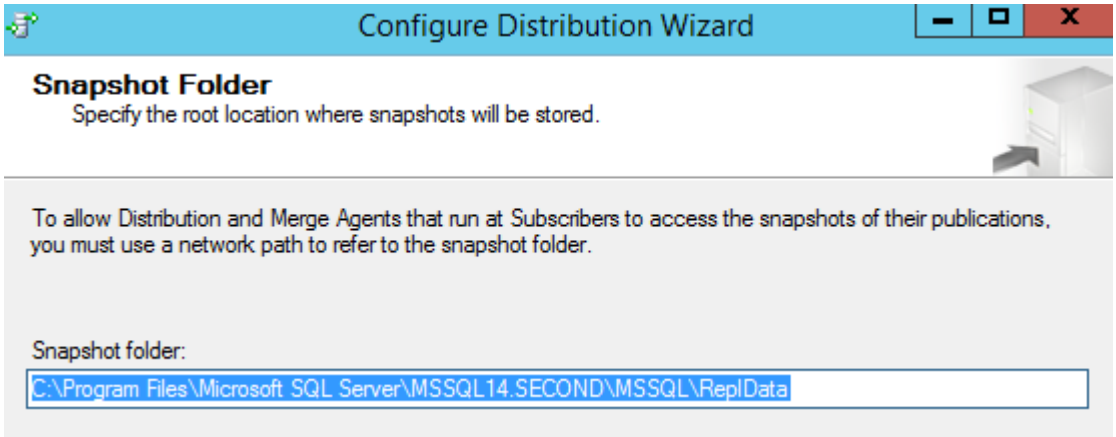
1. Start the SECOND and THIRD instances and the SQL Server Agent on SECOND
2. Reset the replication configuration on PRIM by selecting Disable publishing and distribution from the Replication menu
3. In the pop-up menu of Replication on the SECOND instance select Configure Distribution, and accept the first choice. This will create the distribution database on SECOND.



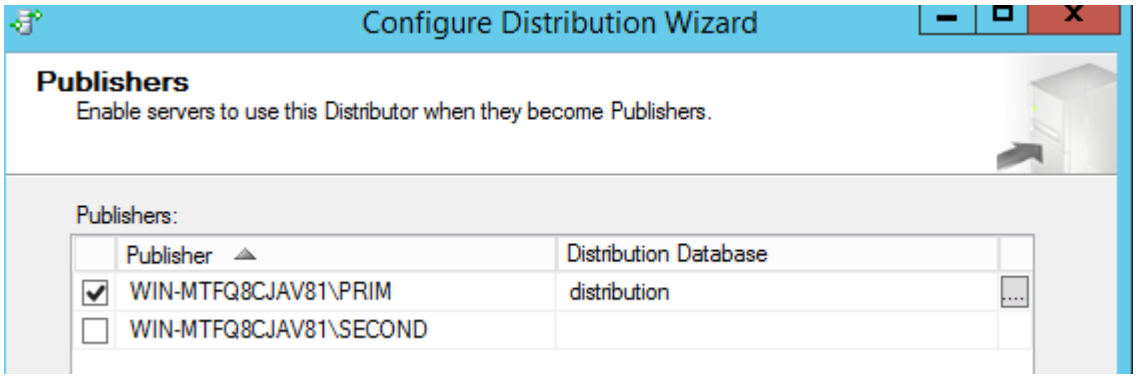
[2]Due to a bug in SQL Server 2019 on Windows 10 systems, the wizard using sp_adddistributor for a remote Distributor uses an empty password instead of the specified one and returns the exception 21768 („The password specified for the @password parameter must be the same when the procedure is executed at the Publisher and at the Distributor"). For this demo to run, execute the sp_adddistributor manually, specifying the right password, like this:

```
use master

exec sp_adddistributor @distributor = 'DESKTOP-....\SECOND', @password = 'type password here'
```

4. On the next panel select that we start the Server Agent manually
5. On the next panel accept the location of the snapshot folder.

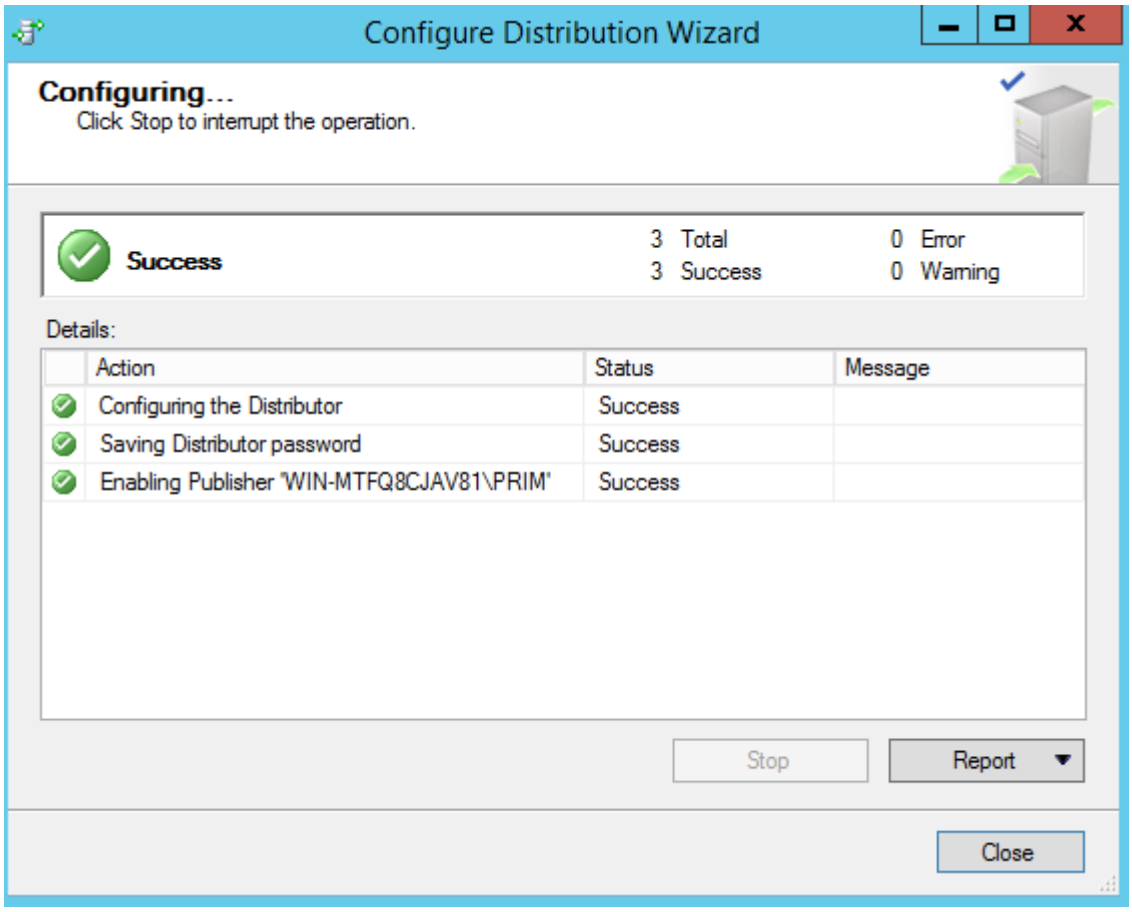


6. On the next panel accept the defaults for the location and name of the distribution database.
7. We then have to specify which publisher servers are allowed to use this distribution database. On the next pane deselect SECOND (since SECOND will not be publishing) and by pressing Add, add the PRIM instance:



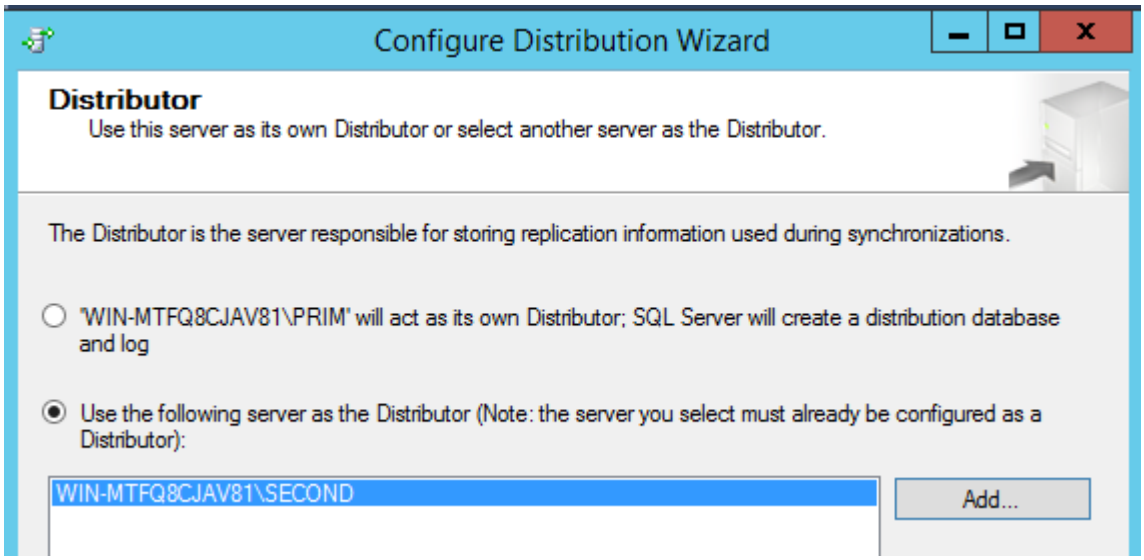
8. On the next panel you must specify a password that the Publishers using this distribution database will need to use. Specify the same password that you use for login. We will need this password later.

9. At the end of the process, you have configured the distributor successfully:



Configuring the publisher

1. In order to configure the PRIM as publisher, we first disable it as distributor. Remember that so far the PRIM acted as its own distributor. In the pop-up menu of Replication select Disable Distribution and Publishing. When the PRIM has been disabled as a distributor, the pop-up menu changes. Select now Configure distribution and specify the SECOND instance as the distributor of PRIM:

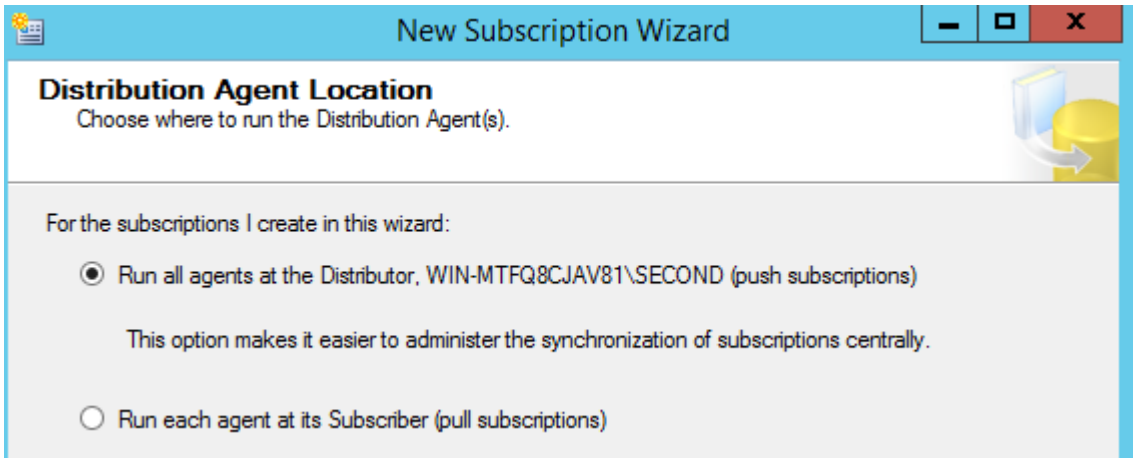


2. On the next pane, enter the same password as before (in Step 8).

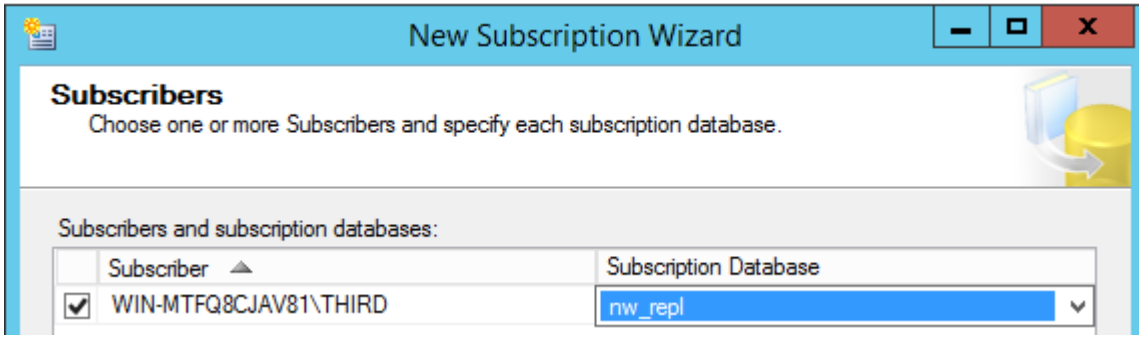
3. The distribution is now set up.

Adding the publication and the subscription

1. Go on creating the transactional publication on PRIM in the usual way (Orders table without any filter).
2. We do not configure the Distribution properties on THIRD because THIRD will not act as a publisher.
3. Create a new subscription on the THIRD instance by selecting Add from Local subscriptions. Select PRIM as the publisher and select the publication that you created in the previous step.
4. Select Run all agents at the Distributor.



5. You can have the subscription database created on THIRD by the wizard as a new database named nw_repl.



6. Test the subscription. Open a query editor on the PRIM and update a record the Orders table. Open a query editor on the THIRD and verify that the change is propagated to the replicated table within 10 seconds.
7. Delete the publication at PRM and the subscription at THIRD. Delete the replicated table on THIRD.

Peer-to-Peer transactional replication

Allows writes to any of the nodes, changes are propagated to all the nodes automatically and only once. Designed for scale-out read load balancing. Concurrent writes of the same record at multiple

nodes i.e. write conflicts are not allowed and treated as a critical error that requires manual resolution[3].

Merge replication

Scenario: our sales employees are traveling and making new orders for their clients. Occasionally they also need to change previous orders when, for example, a shipping address changes. It may also happen that two employees modify the same order. The employees are not continuously connected to the internet. We must design a replication solution that merges all such changes with each other and with the central Northwind database.

In **merge replication**, the duty of the log reader agent is taken by triggers, tables, and views which are created automatically in each subscriber database and also in the publisher database. The triggers log changes in special system tables named 'MSmerge_*' that are created in the same database as the replicated table. There are three triggers named 'MSmerge_[ins, upd, del]_*' created for each table. There are also database level *schema triggers* that log the changes in the schema of the replicated tables.

Merge replication starts with an initial snapshot created by a snapshot agent. By default, the snapshot agent is configured to run every 14 days. Then the duty of the merge agent is similar to the distribution agent in transactional replication with the difference that merge replication is by default configured bi-directional. This means that the agent applies changes on the subscriber and publisher sides as well. There is a separate merge agent for each subscription. The merge agent runs on the distributor in case of a push subscription and on the subscriber in case of a pull subscription.

The distribution database stores only history and error information.

In order to support the bi-directional data synchronization, articles in merge replication must have a uniqueidentifier (GUID) type column that is similar to the identity data type but it produces globally unique IDs[4]. If such a column does not exist, it is added automatically to the tables—which can potentially crash the legacy applications already using the table. The new field is removed when the publication is deleted.

The details of the synch process are not included in this text[5].

The publication

DEMO: In order to develop the solution, follow the steps below.

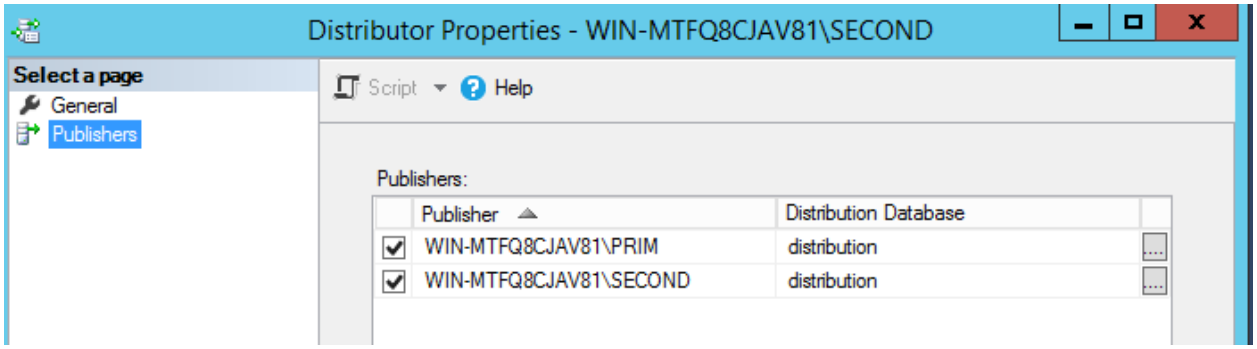
1. Set the SECOND instance as publisher even if it will not publish any publications, otherwise the list of resolvers will not be available:

---

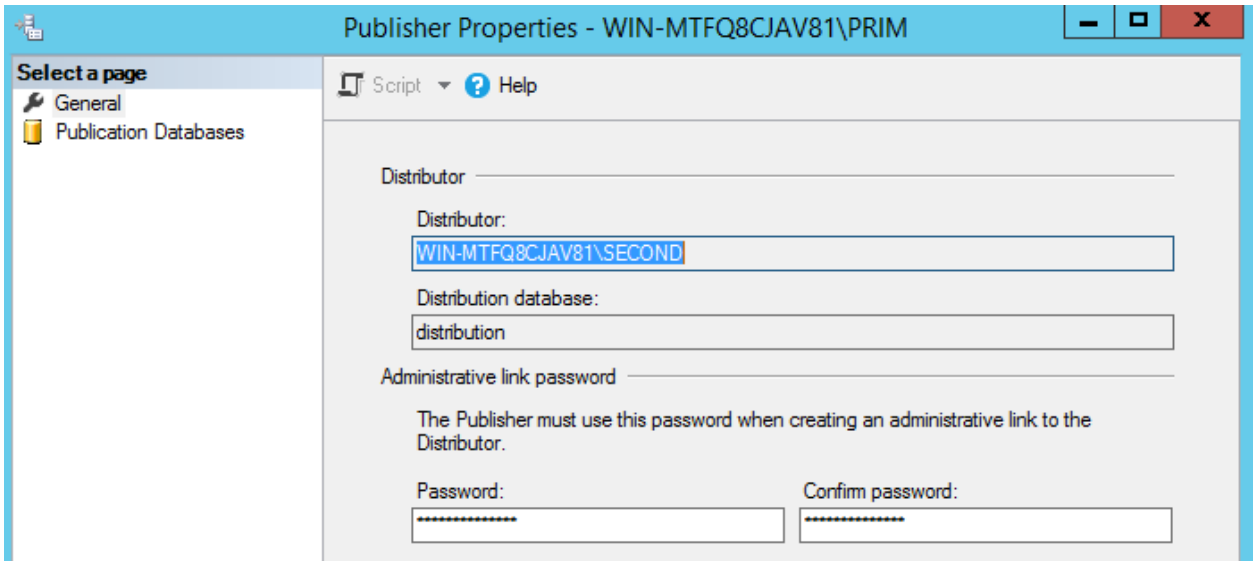
[3] https://learn.microsoft.com/en-us/sql/relational-databases/replication/transactional/peer-to-peer-transactional-replication

[4] Use this data type like this: CREATE TABLE test (my_guid uniqueidentifier DEFAULT NEWSEQUENTIALID() ROWGUIDCOL,… etc

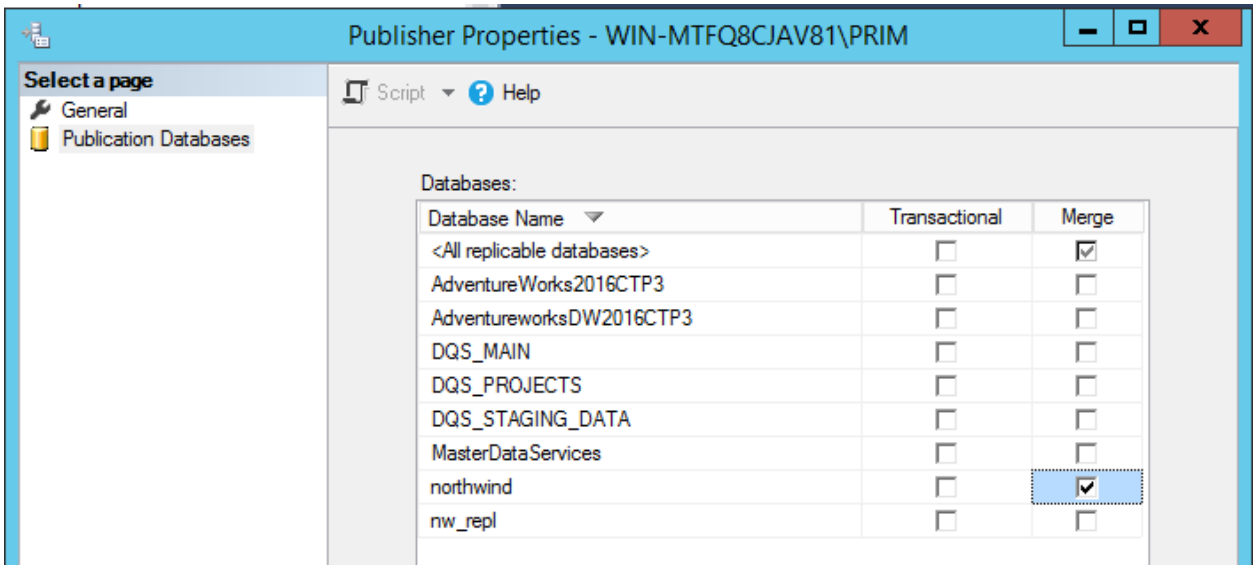
[5] https://documentation.help/replsql/repltypes_30z7.htm



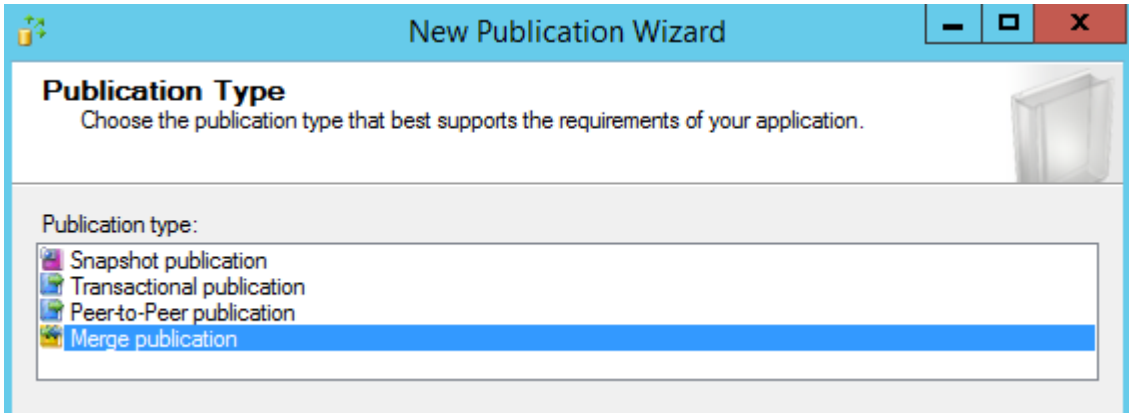
2. Configure the SECOND instance as the distributor of the PRIM (see the previous section). After that, this is what you should see on the PRIM:



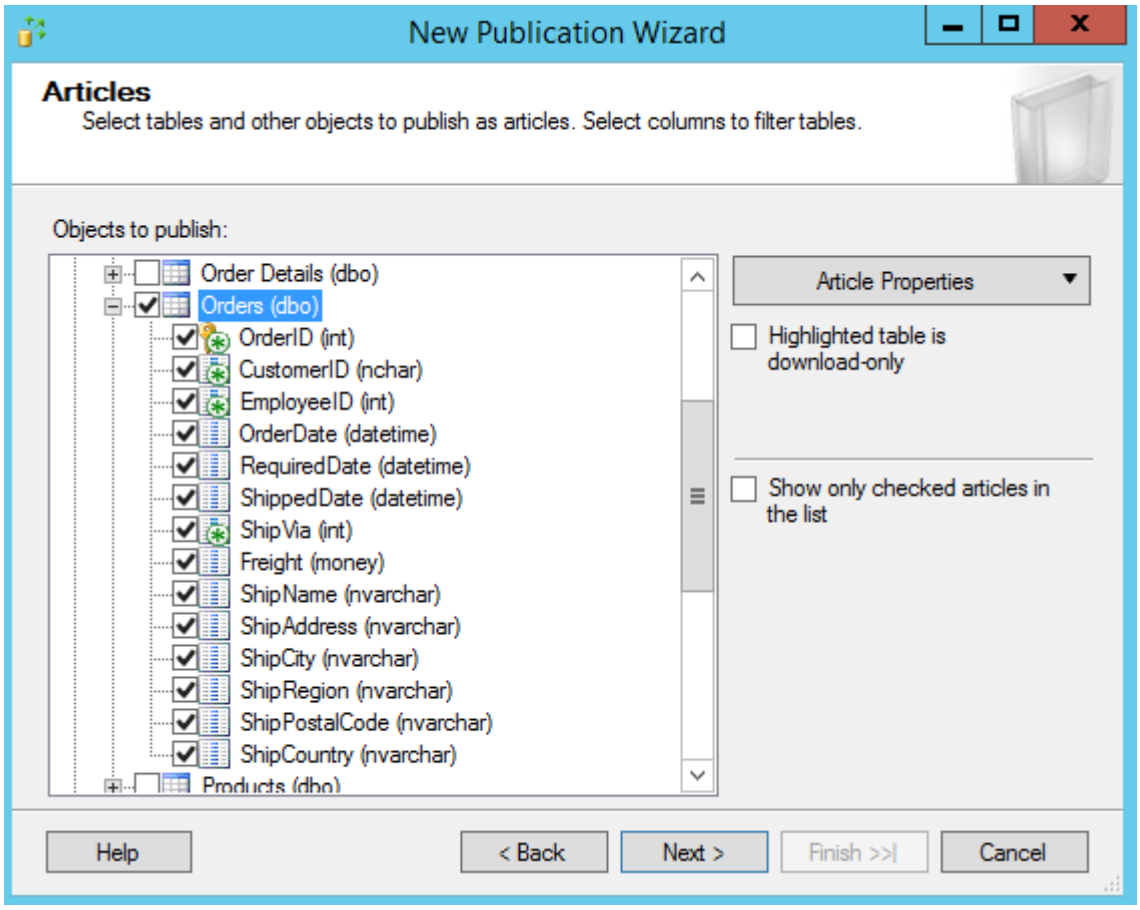
3. Enable the northwind database for merge replication. Select the Publication Databases tab on the publisher (PRIM):



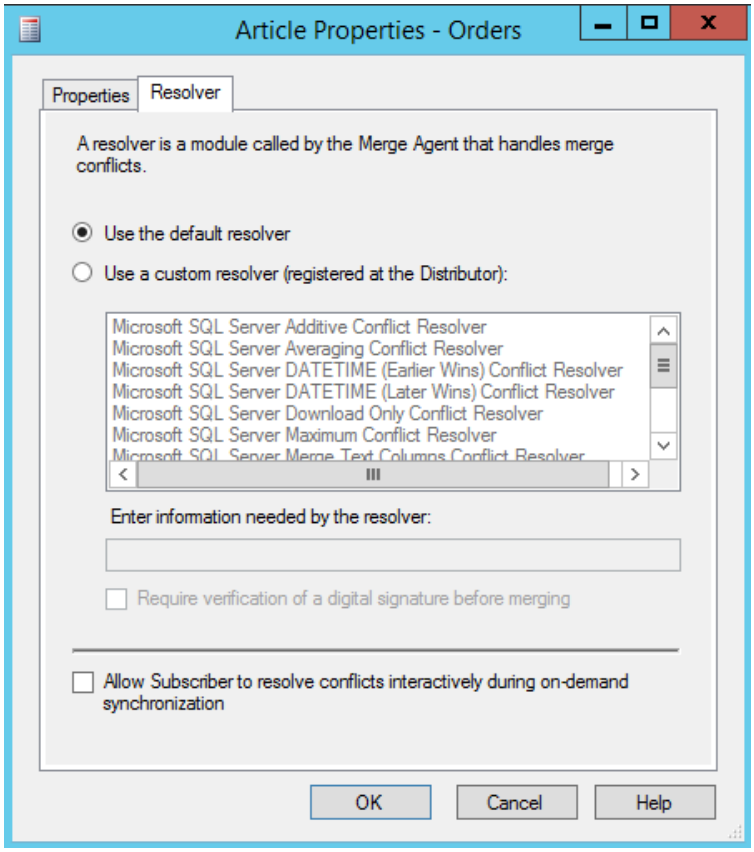
4. Select the northwind database for the publication database and select the publication type:



5. Accept the default subscriber types and select the Orders table as the single article of the publication.



6. You can set various properties for each article by selecting Article Properties, including the resolver to be used for various conflict types. A conflict occurs when an agent tries to change a record that has a pending change. The built-in resolver modules currently registered at the server can be listed by the sp_enumcustomresolvers procedure. For the required parameters of the built-in resolvers, see[6]. You could also add your own resolver as a stored procedure or DLL[7]. The default resolver implements the 'first to Publisher wins' strategy:
    a. if the conflict occurs between a Publisher and a Subscriber, the Publisher change is accepted and the Subscriber value is refused.
    b. if the conflict occurs between two Subscribers using pull subscriptions, then the change from the first Subscriber to synchronize with the Publisher is accepted, and others are refused.

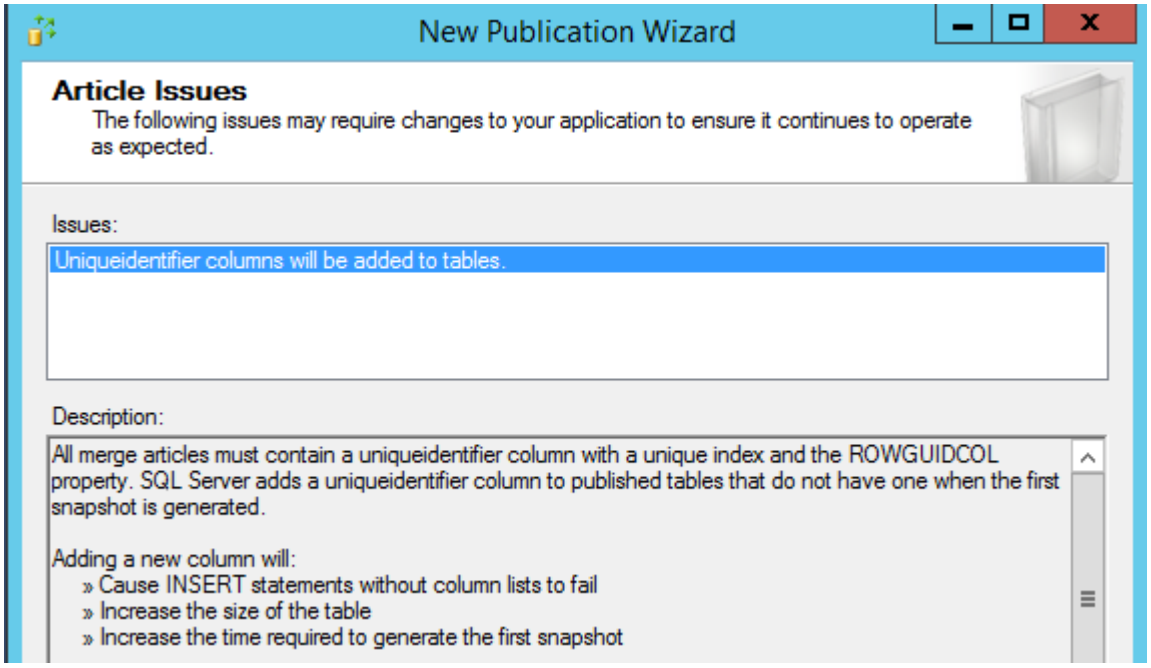


7. The next panel warns that a new GUID will be added to the table. This does not change the primary and foreign key constraints of the table.

[6] https://learn.microsoft.com/en-us/sql/relational-databases/replication/merge/advanced-merge-replication-conflict-com-based-resolvers?view=sql-server-ver16

[7] https://learn.microsoft.com/en-us/sql/relational-databases/replication/implement-a-custom-conflict-resolver-for-a-merge-article

The custom resolver procedure is run on the Publisher and it is called by the Merge agent. It queries the changed record from the Subscriber using the row GUID received from the Merge agent, and returns a result set with a single record that has the same fields as the base table and that contains the values for the winning version of the record.

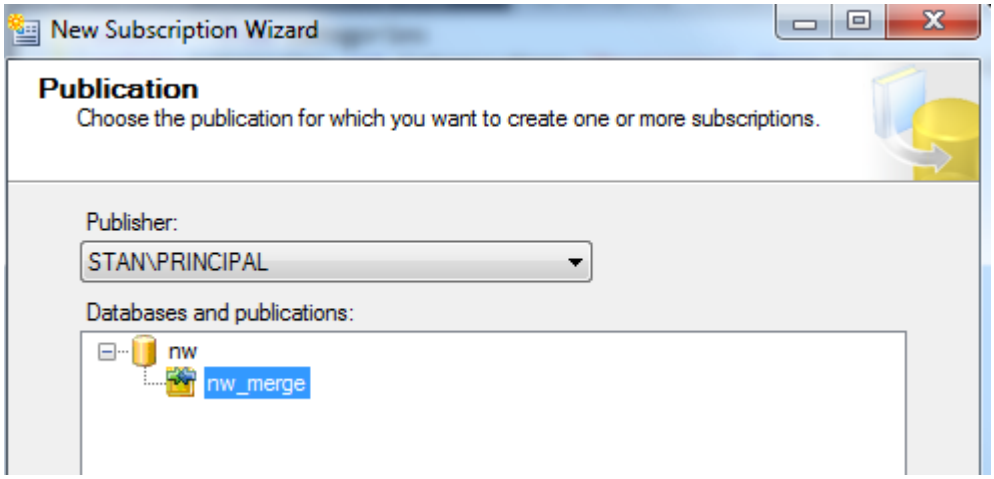


8. For the snapshot agent scheduling, accept the defaults, then define the agent security in the usual way.
9. Name the publication nw_merge and create it. Check the new GUID column and the 3 new triggers on the Orders table.

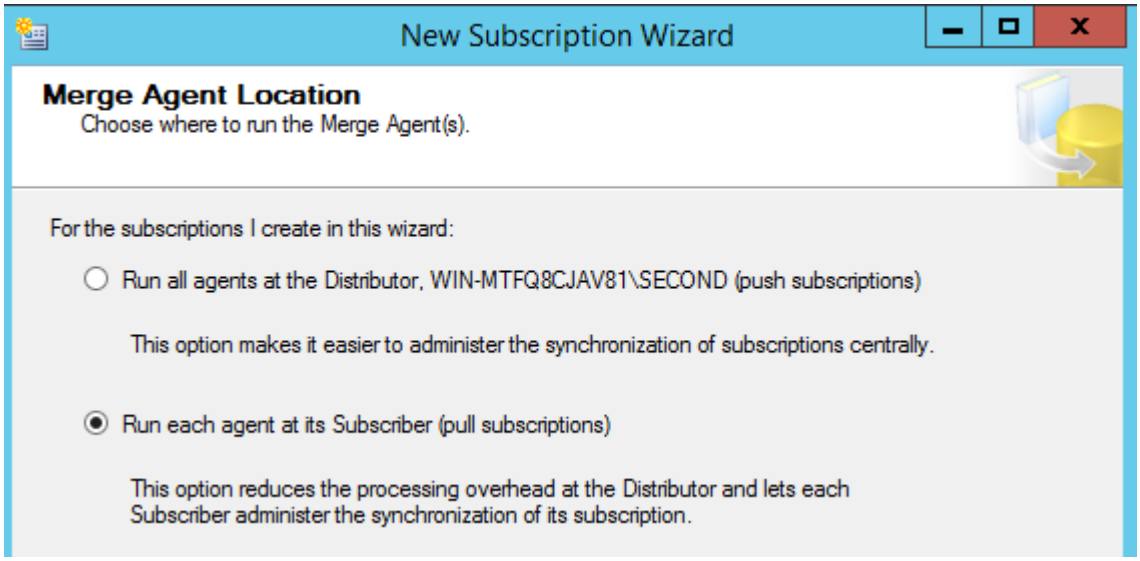
The subscription

We add two subscribers to see the merge process in action.

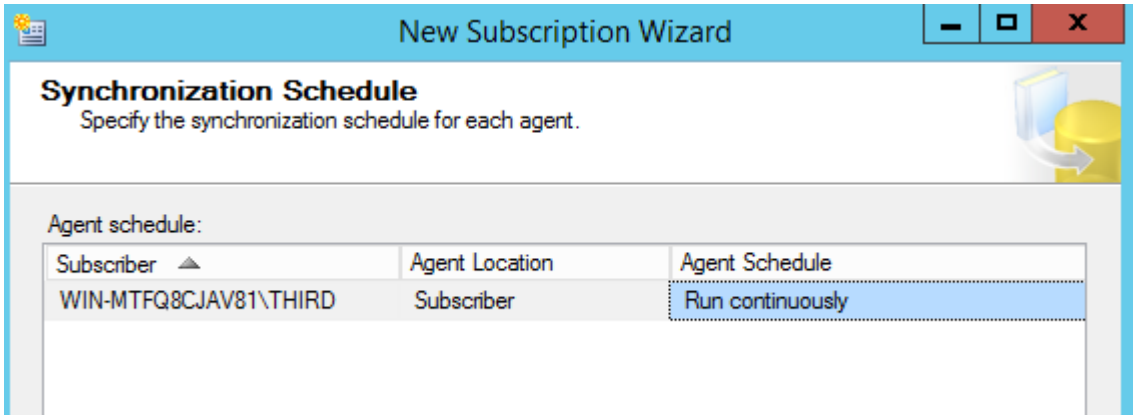
1. Start the SQL Server Agent on THIRD manually
2. On the THIRD instance, select New subscription and specify the PRIM as the publisher:



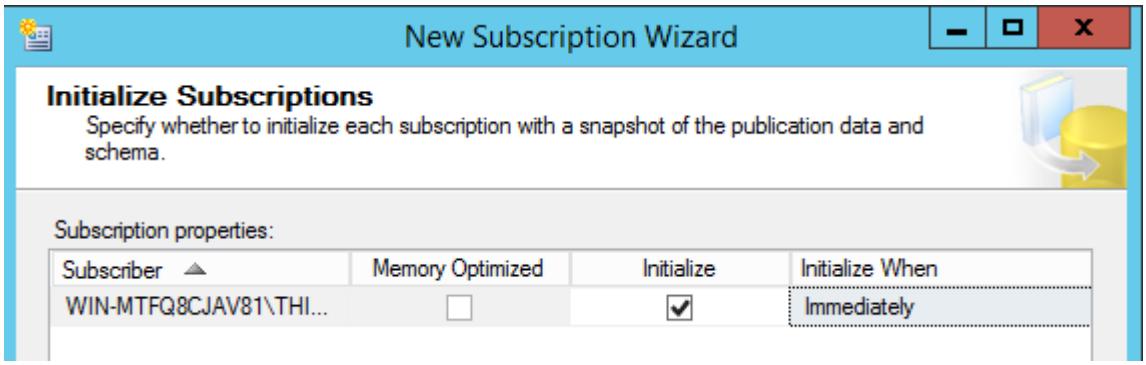
3. On the next pane, select pull subscription. This is in line with the usual expectation that the subscribers will want to schedule their synchronization.



4. Add the two subscribers, one on the THIRD instance and one on the PRIM, with newly created databases named nw_merge_1-2.
5. Set the security on the agents in the usual way.
6. We set the synch schedule for both merge agents to 'Run continuously'. *Note: this setting is only for test and demo purposes. In a real life scenario, the merge process would start either at specified intervals or on demand, when a specific event, e.g. a VPN connection occurs on the subscriber.*



7. Initialize the subscriptions immediately.



8. In merge publication, the subscribers may be allowed to republish the publication to which they are subscribed (Server type subscription), thus creating a hierarchical subscribing architecture. A Server type subscription also allows the assignment of an explicit numeric priority to each server that will be used in case of a conflict (see later). Select Client type subscription.

9. Finalize and start the replication.
10. Verify that the changed values are propagated from any subscriber or the publisher to all the other by editing the Orders table in three separate editors. Be patient: It can take up to 2 minutes for the synchronization to complete[8].
11. Test the first-come-first-served style default conflict resolution when the two subscribers update a record 'simultaneously' (considering that the agents poll the logs once a minute). The change request that reaches first the Publisher database will be applied to all parties. Use the Window -> Add horizontal tab group command in SSMS to see all three tables simultaneously. You can refresh the data grids by pressing Ctl-R.

[8] Distributed database systems may be inconsistent at any time of observation, but the synch mechanism guarantees that if no further event occurs, a global consistency (consensus) will be reached after some time. This behaviour is called **eventual consistency**. According to the CAP theorem, the C, A, P properties cannot be all satisfied fully in any distributed database system: Consistency (all reads receive the most recent write or an error), Availability (all reads contain data, but it might not be the most recent), Partition tolerance (the system tolerates network failures) -> since network failures do occur, consistency and availability must be traded-off depending on application requirement s. See https://www.bmc.com/blogs/cap-theorem/

WIN-MTFQ8CJAV81\...ind - dbo.Orders

OrderID	CustomerID	EmployeeID	OrderDate
10248	HANAR	1	1996-07-04 00:0...
10249	HANAR	6	1996-07-05 00:0...
10250	HANAR	4	1996-07-08 00:0...

1 of 200 Cell is Read Only.

WIN-MTFQ8CJAV81\..._m2 - dbo.Orders

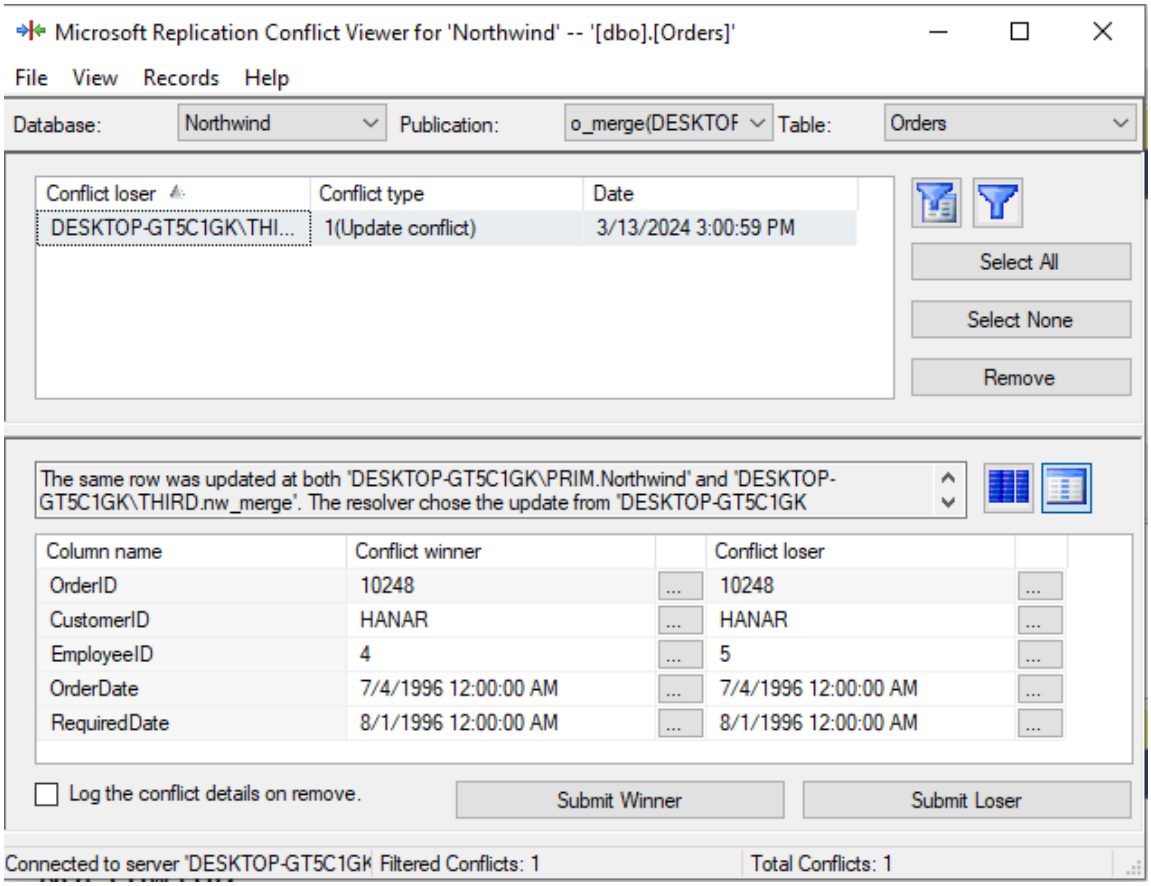
OrderID	CustomerID	EmployeeID	OrderDate
10248	HANAR	2	1996-07-04 00:0...
10249	HANAR	6	1996-07-05 00:0...
10250	HANAR	4	1996-07-08 00:0...

1 of 200 Cell is Modified.

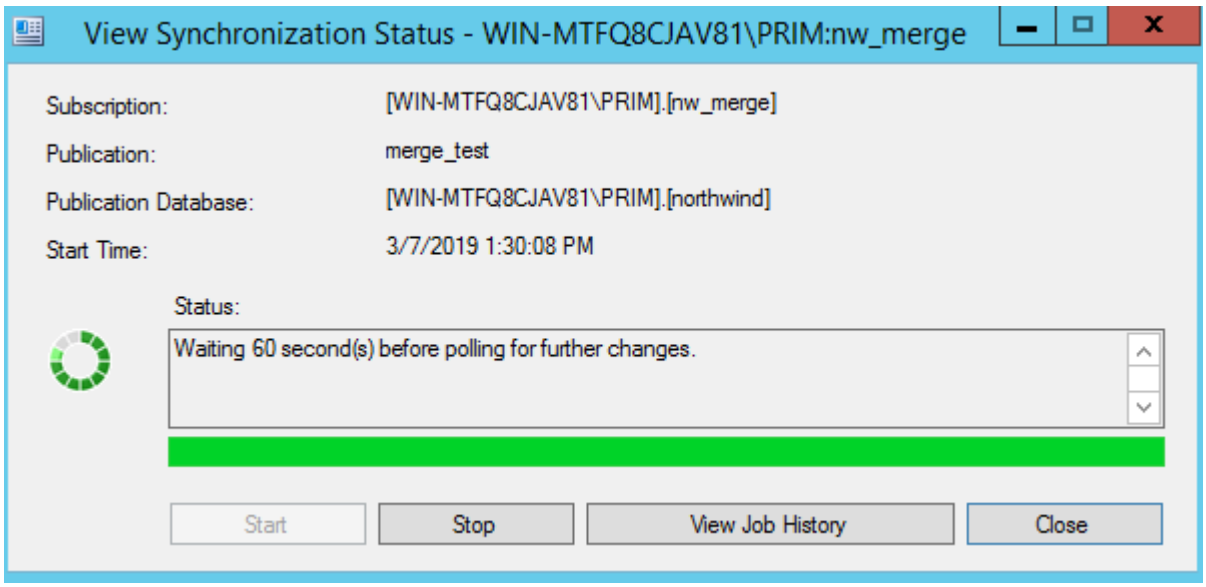
WIN-MTFQ8CJAV81\...ger2 - dbo.Orders

OrderID	CustomerID	EmployeeID	OrderDate
10248	HANAR	3	1996-07-04 00:0...
10249	HANAR	6	1996-07-05 00:0...
10250	HANAR	4	1996-07-08 00:0...
10251	VICTE	3	1996-07-08 00:0

12. Merge conflicts can be viewed and manually resolved in the Replication Conflict Viewer tool, which is available from the pop-up menu of the publication. If a record has been updated by all three servers i.e. the conflict involves 3 parties, it will appear as two separate conflicts, with the same winner. Here you can either accept the default resolution (by clicking Submit Winner) or revert it (by clicking Submit Loser):



13. You can review the current state of the merge process by selecting View synch status from the pop-up menu of the subscription. You can also start the process manually.



14. Generate and review replication scripts on both instances by selecting Generate scripts from the pop-up menu of the Replication group.

15. Delete all replication objects.

Log shipping

While the primary application area of replication is business process automation, log shipping is designed to prepare for **disaster recovery** of a database, by creating and synchronizing an exact,

usually read-only copy of the database also known as a 'warm' standby database. A log can be shipped from the primary sever to multiple secondary servers.

Log shipping starts with a full backup of the database which is restored on the client. Then the three steps involved in log shipping are as follows.

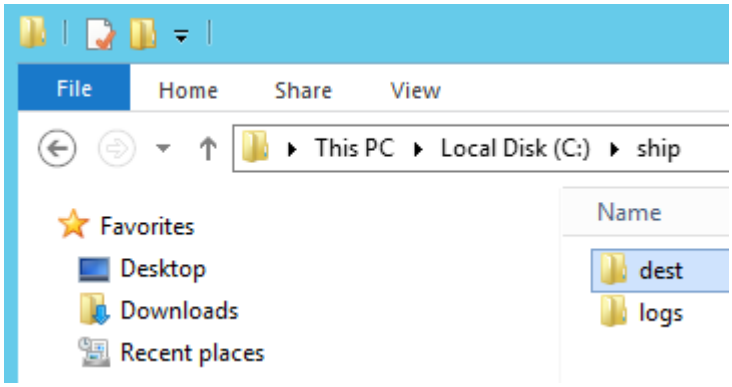
1. A backup job running on the primary server backs up the new portion of the database transaction log to the local server
2. A copy job running on the secondary server copies the log to a configurable destination (e.g. a network file server)
3. A restore job running on the secondary server restores the backup to the secondary database(s)

An alert job may also be running monitoring whether each step is performed as expected and on time.

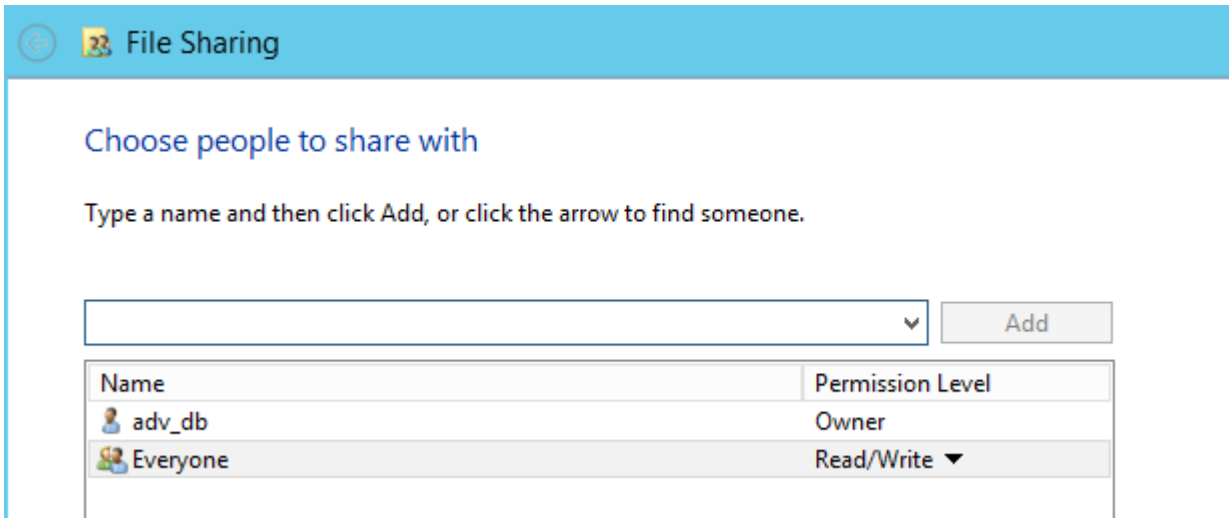
Depending on the scheduling of the jobs, there is a latency between the two databases. This latency can be exploited when the primary database is modified by mistake.

DEMO: In our scenario, we create a '**warm backup'** of the Northwind database as follows. This means that in case of a server failure, the standby server (i.e. the warm backup) can replace the production server in a couple of minutes, without a long full restore procedure. The primary and secondary servers run normally on different server machines, but in our demo we will use the same machine and two server instances, the Primary and the Secondary.

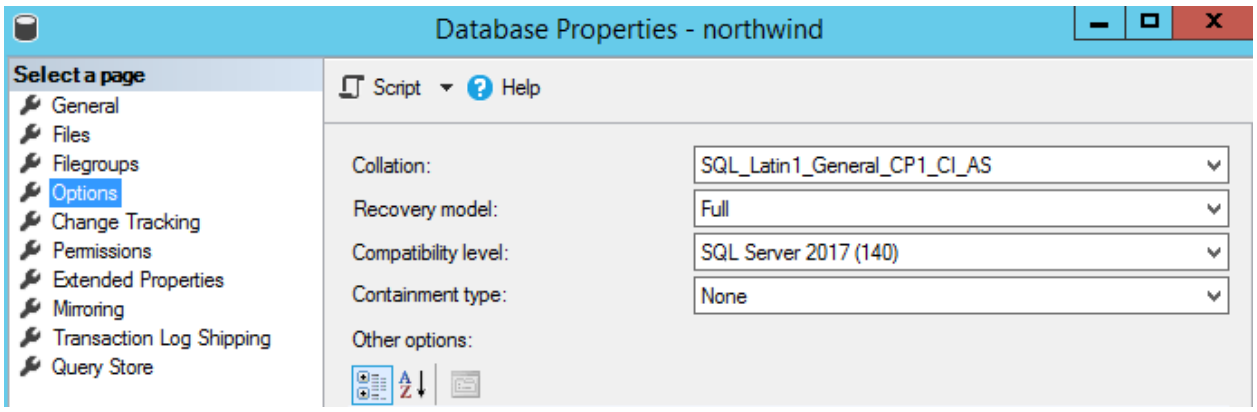
1. Create a folder for the storing of the logs and another where the copy will be placed.



2. Set sharing for *both* folders to shared to everyone (Properties/Sharing/Share). Type Everyone and set Read/Write.

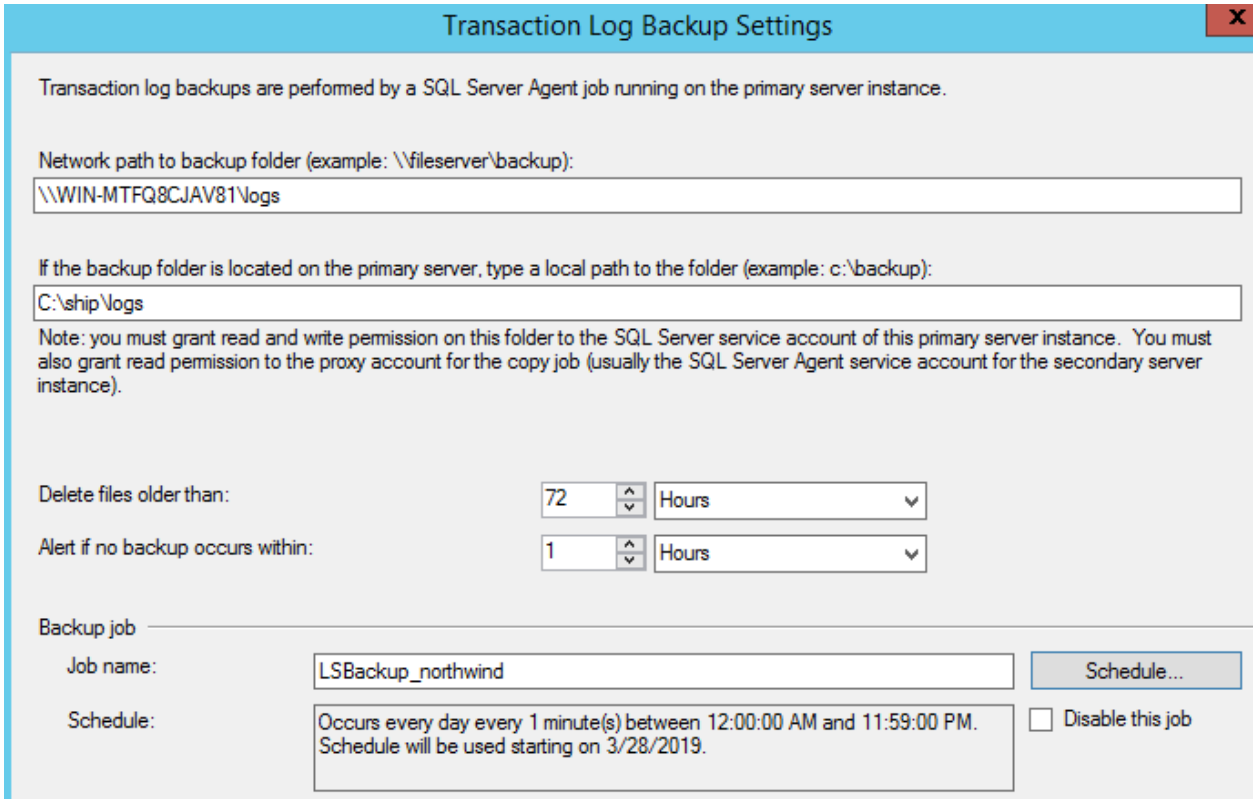


3. In the pop-up menu of the Northwind database on the Primary server, select Properties and verify that the recovery model of the Northwind database is set to Full. This means that the inactive parts of the log will not be deleted at a checkpoint.

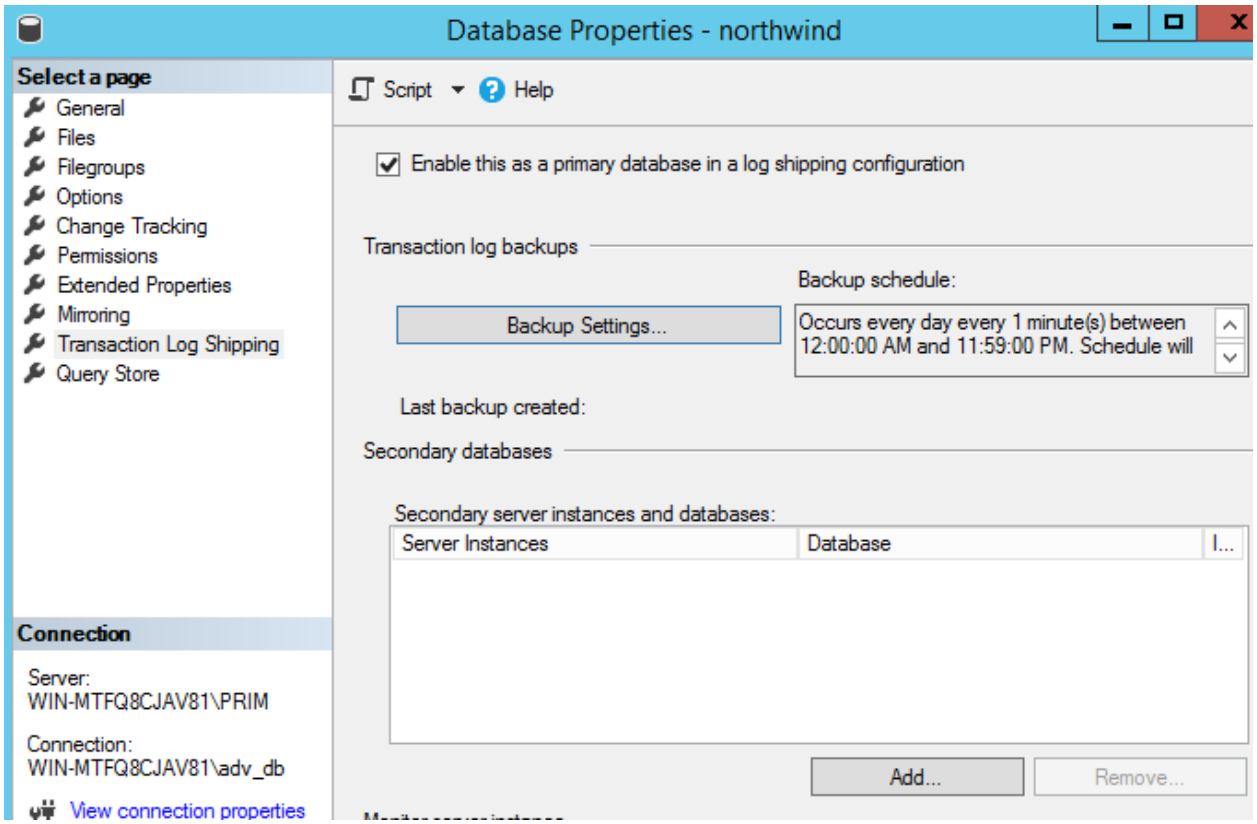


4. On the PRIM, in the Nortwind database Tasks -> Ship Transaction logs panel, enable the database for shipping, and select Backup settings. Type \\WIN-MTFQ8CJAV81\logs as the network path of the backup folder and C:\ship\logs as the local folder path.

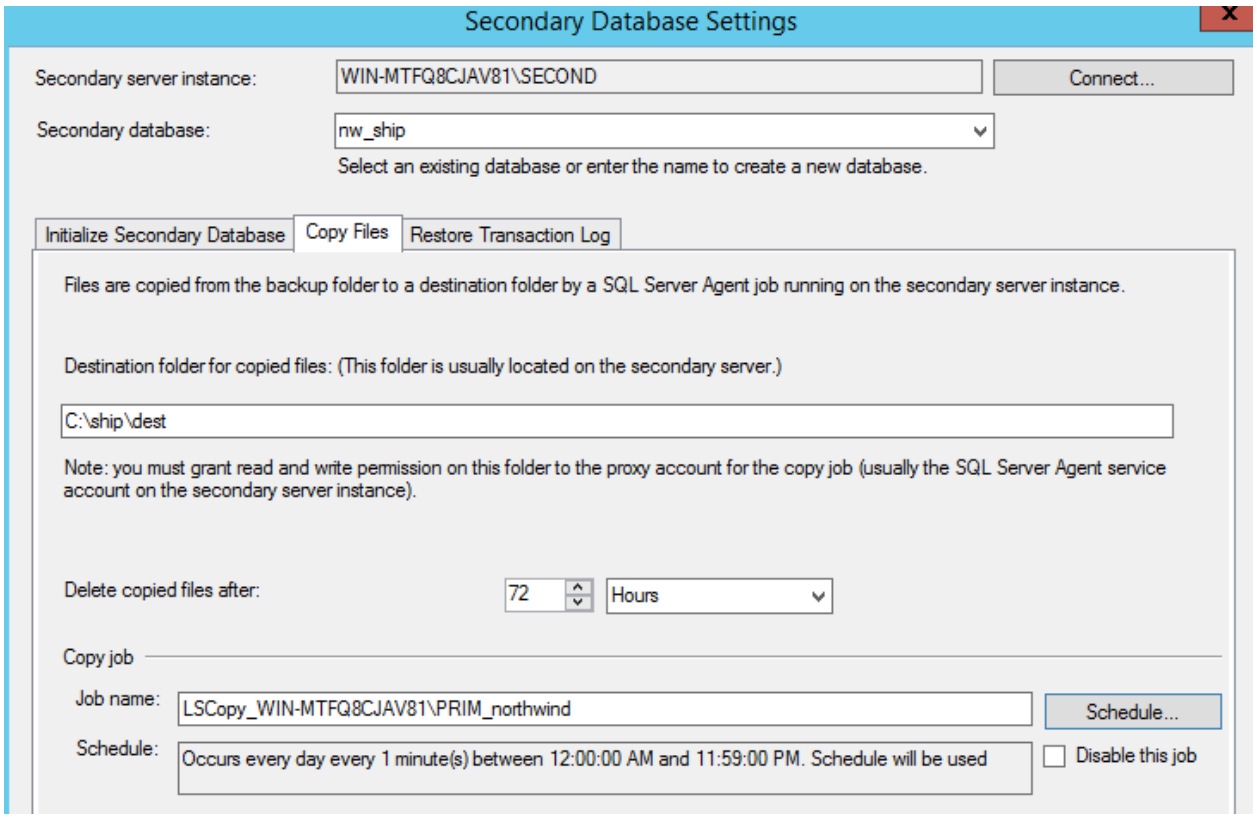
5. Select Edit job and set a schedule that runs the job every minute. *Note: this setting is only for demo purposes.*



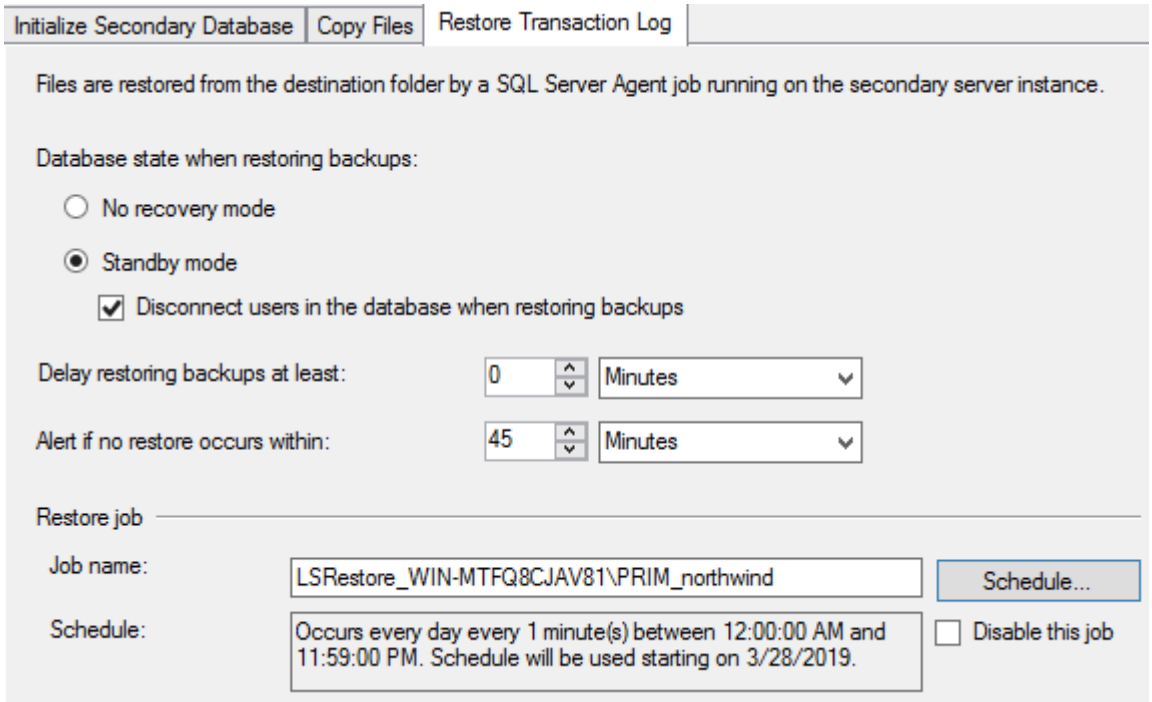
6. Return to the Database properties panel and select Add in the Secondary databases section:



7. On the Secondary database settings panel, type the new database name nw_ship. Accept the default for initialization. In the Copy files tab, enter C:\ship\dest as the destination path. Set the schedule for every 1 minute. *Note: this setting is only for demo purposes*.



8. On the Restore tab, select Standby mode and Disconnect users and also specify a restore schedule of every 1 minute. *Note: this setting is only for demo purposes*.



9. Optionally, you can configure the THIRD server as a Monitor server. The Monitor server is the instance which monitors both the primary and secondary servers and where the log shipping alert job runs. This job generates an error if either of the three processes (backup, copy, restore) is not performed successfully for a time period longer than the preset threshold (which defaults to 45 minutes). Use a shorter period, e.g. 2 minutes for testing purposes. Use the sa SQL Server principal to authenticate the alert job to access the primary and secondary instances. Also, two alerts will automatically be setup on the monitor instance, for the failure of the primary and the secondary instances, respectively, but with empty responses (i.e. to be configured by the administrator). A typical response would be sending an email to an operator.
10. Finalize and run the configuration.
11. Verify the correct operation by connecting to both databases. You may need to wait 3 minutes to see the changes in the secondary database. You can generate a report of the current status of log shipping from the popup menu of the monitor server -> Reports -> Standard Reports -> Transaction Log Shipping Status.
12. Check the contents of the log and dest folders. You should see the transaction log backups appear every minute.
13. Disable the log shipping on the PRIM server. Since the shipped database on Secondary is in warm standby state, you must execute the following commands before you can drop it (see below). Alternatively, you can set the database to single user mode on the SSMS GUI, restore the database with recovery, and set it back to multi-user.

```
use master
alter database nw_ship set single_user with rollback immediate
restore database nw_ship with recovery
alter database nw_ship set multi_user
```

Failover clusters

According to a definition, "Failover clustering (FC) is a Windows Server feature that enables you to group multiple servers together into a fault-tolerant cluster to increase availability and scalability of applications and services such as … Microsoft SQL Server"[9]. A FC requires a server type operating system running on more than one server machines. This is one of the reasons why we cannot demonstrate this technology in this course.

Once the FC has been setup, SQL Server instances running on the cluster members can be configured into an Always On Availability Cluster. In such a cluster, we can define a group of databases a.k.a. 'availability databases' that are copied to other instances and that fail over together. Besides failover support, such an architecture can be configured for automatic read load balancing for heavy load databases as well.

[9] https://docs.microsoft.com/hu-hu/windows-server/manage/windows-admin-center/use/manage-failover-clusters
https://docs.microsoft.com/hu-hu/windows-server/failover-clustering/failover-clustering-overview